

Vertex AI Pipelines PSC 介面明確 Proxy

來源：<https://codelabs.developers.google.com/pipelines-psc-interface-proxy?hl=zh-tw>

步驟數：18

在本教學課程中，您將瞭解如何透過 Vertex AI Pipelines 設定及驗證 Private Service Connect 介面。

目錄

1. 簡介
2. 事前準備
3. 消費者設定
4. 啟用 IAP
5. 建立消費者 VM 執行個體
6. Private Service Connect 網路連結
7. 私人 DNS 區域
8. 更新明確 Proxy
9. 建立 Jupyter Notebook
10. 建立 Vertex AI Workbench 執行個體
11. 更新 Vertex AI 服務代理
12. 啟用 Tcpdump
13. 部署 Vertex AI Pipelines 工作
14. PSC 介面驗證
15. Cloud Logging 驗證
16. TCPDump 驗證
17. 清理
18. 恭喜

步驟 1 · 約 0 分鐘

Proxy Vertex AI Pipelines PSC 介面明確 Proxy

程式碼研究室簡介

subject上次更新時間：3月 27, 2026

account_circle作者：Deepak Michael, Zhihan Xu

1. 簡介

Private Service Connect 介面是一種資源，可讓供應商虛擬私有雲 (VPC) 網路啟動與用戶虛擬私有雲網路中各種目的地的連線。供應商和用戶網路可以位於不同專案和機構中。

如果網路連結接受來自 Private Service Connect 介面的連線，Google Cloud 會從網路連結指定的消費者子網路，為介面分配 IP 位址。消費者和生產者網路已連線，可使用內部 IP 位址通訊。

網路連結與 Private Service Connect 介面之間的連線，類似於 Private Service Connect 端點與服務連結之間的連線，但有兩項主要差異：

- 網路連結可讓供應商網路啟動與消費者網路的連線 (代管服務輸出)，而端點則可讓消費者網路啟動與供應商網路的連線 (代管服務輸入)。
- Private Service Connect 介面連線是可遞移的。也就是說，生產端網路可以與連線至消費端網路的其他網路通訊。

Vertex AI PSC 介面可連線性考量

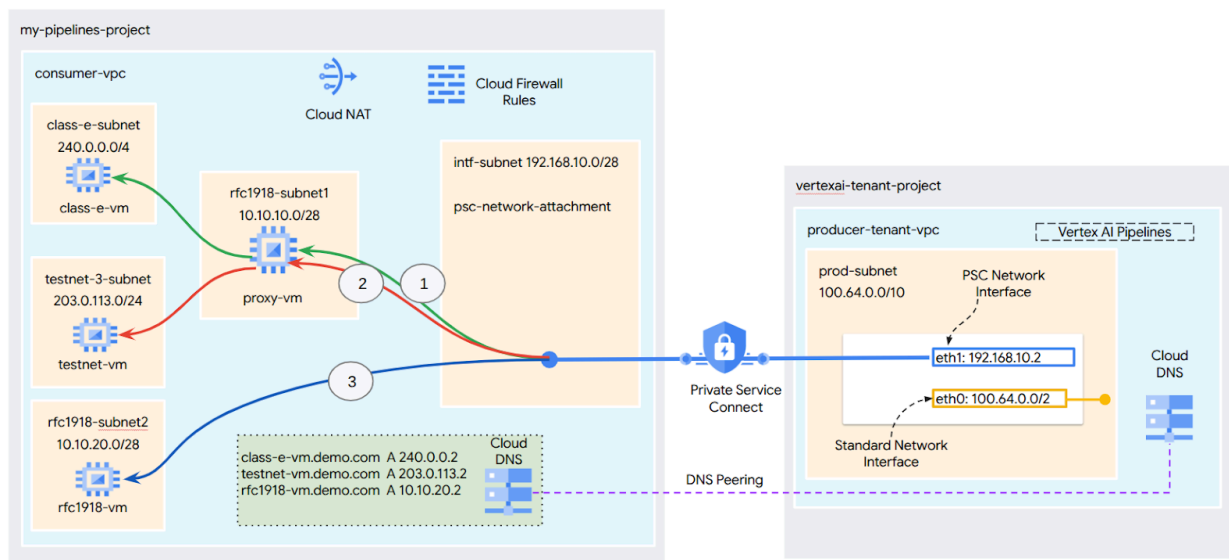
- PSC 介面可將流量轉送至 RFC1918 位址區塊內的 VPC 或內部部署目的地。
- 如果 PSC 介面指定非 RFC-1918 位址區塊，則必須在消費者 VPC 中部署具有 RFC-1918 位址的明確 Proxy。在 Vertex AI 部署作業中，必須定義 Proxy，以及目標端點的 FQDN。請參閱圖 1，瞭解在客戶 VPC 中設定的明確 Proxy，方便將流量路由至下列非 RFC-1918 CIDR：

[1] 240.0.0.0/4

[2] 203.0.113.0/2

[3] 10.10.20.0/28 不需要 Proxy，屬於 rfc1918 範圍。

- 如果您只使用 PSC 介面設定部署作業，系統會保留預設的網際網路存取權。這類傳出流量會直接從 Google 代管的安全租戶網路輸出。



Vertex AI PSC 介面 VPC-SC 注意事項

- 如果專案屬於 VPC Service Controls 範圍，範圍會封鎖 Google 代管的租戶預設網際網路存取權，防止資料竊取。
- 如要允許部署作業在此情境中存取公開網際網路，您必須明確設定安全輸出路徑，透過 VPC 傳送流量。建議您在 VPC 邊界內設定 RFC1918 位址的 Proxy 伺服器，並建立 Cloud NAT 閘道，允許 Proxy VM 存取網際網路。

注意：使用 VPC-SC 時，Vertex AI Pipelines 部署作業需要明確的網際網路輸出 Proxy。如果未啟用 VPC-SC，系統會透過 Google 管理的租戶 VPC 提供網際網路輸出流量。

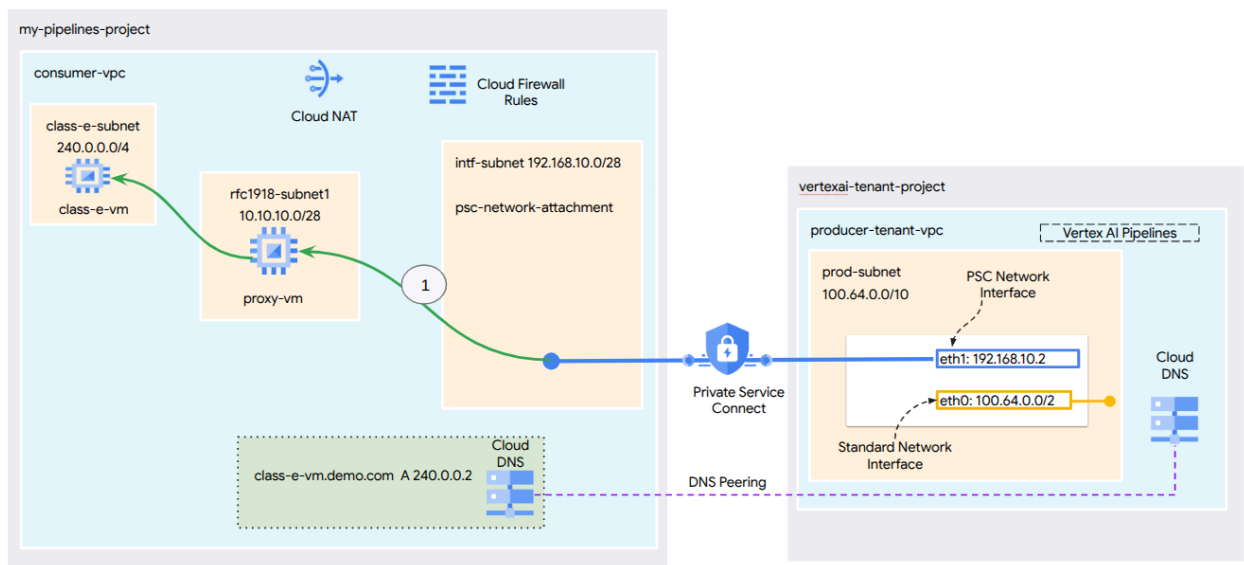
如需更多資訊，請參閱下列資源：

為 [Vertex AI 資源設定 Private Service Connect 介面](#) | [Google Cloud](#)

建構項目

在本教學課程中，您將建構完整的 Vertex AI Pipelines 部署作業，並使用 Private Service Connect (PSC) 介面，允許從生產者連線至消費者的運算資源，如圖 1 所示，目標為非 rfc-1928 端點。

圖 2



您會在用戶 VPC 中建立單一 psc-network-attachment，利用 DNS 對接來解析代管 Vertex AI 訓練 的租戶專案中的用戶 VM，進而實現下列用途：

1. 部署 Vertex AI Pipelines，並設定 Proxy VM 做為明確的 Proxy，允許對 Class E 子網路中的 VM 執行 wget。

注意：本教學課程會根據圖示拓撲提供設定和驗證步驟，請視貴機構的需求修改程序。

課程內容

- 如何建立網路連結
- 生產者如何使用網路連結建立 PSC 介面
- 如何使用 DNS 對接，建立從生產端到消費端的通訊
- 如何從 Vertex AI Pipelines 建立與非 RFC1918 IP 位址空間的通訊

軟硬體需求

Google Cloud 專案

IAM 權限

- [Compute 網路管理員](#) (roles/compute.networkAdmin)
 - [Compute 執行個體管理員](#) (roles/compute.instanceAdmin)
 - [Compute 安全管理員](#) (roles/compute.securityAdmin)
 - [DNS 管理員](#) (roles/dns.admin)
 - [受 IAP 保護的通道使用者](#) (roles/iap.tunnelResourceAccessor)
 - [記錄管理員](#) (roles/logging.admin)
 - [Notebooks 管理員](#) (roles/notebooks.admin)
 - [專案 IAM 管理員](#) (roles/resourcemanager.projectIamAdmin)
 - [服務帳戶管理員](#) (roles/iam.serviceAccountAdmin)
 - [服務使用情形管理員](#) (roles/serviceusage.serviceUsageAdmin)
-

步驟 2 · 約 2 分鐘

2. 事前準備

更新專案以支援教學課程

本教學課程會使用 `$variables`，協助您在 Cloud Shell 中實作 `gcloud` 設定。

在 Cloud Shell 中執行下列操作：

```
gcloud config list project
gcloud config set project [YOUR-PROJECT-NAME]
projectid=YOUR-PROJECT-NAME
echo $projectid
```

啟用 API

在 Cloud Shell 中執行下列操作：

```
gcloud services enable "compute.googleapis.com"
gcloud services enable "aiplatform.googleapis.com"
gcloud services enable "dns.googleapis.com"
gcloud services enable "notebooks.googleapis.com"
gcloud services enable "storage.googleapis.com"
gcloud services enable "cloudresourcemanager.googleapis.com"
gcloud services enable "artifactregistry.googleapis.com"
gcloud services enable "cloudbuild.googleapis.com"
```

步驟 3 · 約 10 分鐘

3. 消費者設定

建立 Consumer VPC

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks create consumer-vpc --project=$projectid --subnet-mode=custom
```

建立消費者子網路

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks subnets create class-e-subnet --project=$projectid --range=240.0.0.0/4 --network=consumer-vpc --region=us-central1
```

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks subnets create rfc1918-subnet1 --project=$projectid --range=10.10.10.0/28 --network=consumer-vpc --region=us-central1
```

建立 Private Service Connect 網路連結子網路

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks subnets create intf-subnet --project=$projectid --range=192.168.10.0/28 --network=consumer-vpc --region=us-central1
```

Cloud Router 和 NAT 設定

在本教學課程中，Cloud NAT 用於為沒有公開 IP 位址的 Proxy VM 提供網際網路存取權。有了 Cloud NAT，只有私人 IP 位址的 VM 也能連上網際網路，執行安裝軟體套件等工作。

在 Cloud Shell 中建立 Cloud Router。

```
gcloud compute routers create cloud-router-for-nat --network consumer-vpc --region us-central1
```

在 Cloud Shell 中建立 NAT 閘道。

```
gcloud compute routers nats create cloud-nat-us-central1 --router=cloud-router-for-nat --auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges --region us-central1 --enable-logging --log-filter=ALL
```

步驟 4 · 約 5 分鐘

4. 啟用 IAP

如要允許 IAP 連線至您的 VM 執行個體，請根據以下條件建立防火牆規則：

- 套用至所有您希望能透過 IAP 存取的 VM 執行個體。
- 允許來自 IP 範圍 35.235.240.0/20 的輸入流量。這個範圍包含 IAP 用於 TCP 轉送的所有 IP 位址。

在 Cloud Shell 中，建立 IAP 防火牆規則。

```
gcloud compute firewall-rules create ssh-iap-consumer \  
  --network consumer-vpc \  
  --allow tcp:22 \  
  --source-ranges=35.235.240.0/20
```

步驟 5 · 約 0 分鐘

5. 建立消費者 VM 執行個體

在 Cloud Shell 中，建立消費者 VM 執行個體 class-e-vm。

```
gcloud compute instances create class-e-vm \
  --project=$projectid \
  --machine-type=e2-micro \
  --image-family debian-11 \
  --no-address \
  --shielded-secure-boot \
  --image-project debian-cloud \
  --zone us-central1-a \
  --subnet=class-e-subnet \
  --metadata startup-script="#!/bin/bash
  sudo apt-get update
  sudo apt-get install tcpdump
  sudo apt-get install apache2 -y
  sudo service apache2 restart
  echo 'Class-e server !!' | tee /var/www/html/index.html
  EOF"
```

在 Cloud Shell 中建立用戶端 VM 執行個體 proxy-vm，做為 Vertex AI Pipelines 的明確 Proxy。我們會使用 tinypoxy 做為轉送 HTTP 流量的應用程式，但系統也支援 HTTPS。

```
gcloud compute instances create proxy-vm \
  --project=$projectid \
  --machine-type=e2-micro \
  --image-family debian-11 \
  --no-address \
  --can-ip-forward \
  --shielded-secure-boot \
  --image-project debian-cloud \
  --zone us-central1-a \
  --subnet=rfc1918-subnet1 \
  --metadata startup-script="#!/bin/bash
  sudo apt-get update
  sudo apt-get install tcpdump
  sudo apt-get install tinypoxy -y
  sudo apt-get install apache2 -y
  sudo service apache2 restart
  echo 'proxy server !!' | tee /var/www/html/index.html
  EOF"
```

步驟 6 · 約 5 分鐘

6. Private Service Connect 網路連結

網路連結是區域資源，代表 Private Service Connect 介面的用戶端。您會將單一子網路與網路連結建立關聯，而生產端會從該子網路將 IP 指派給 Private Service Connect 介面。子網路必須與網路連結位於同一地區。網路連結必須與生產者服務位於相同區域。

建立網路連結

在 Cloud Shell 中建立網路連結。

```
gcloud compute network-attachments create psc-network-attachment \
  --region=us-central1 \
  --connection-preference=ACCEPT_AUTOMATIC \
  --subnets=intf-subnet
```

列出網路連結

在 Cloud Shell 中列出網路連結。

```
gcloud compute network-attachments list
```

說明網路連結

在 Cloud Shell 中，說明網路附件。

```
gcloud compute network-attachments describe psc-network-attachment --region=us-central1
```

請記下 psc-network-attachment 名稱，供應商建立 Private Service Connect 介面時會用到。

如要在 Cloud 控制台中查看 PSC 網路附件網址，請前往下列位置：

「網路服務」→「Private Service Connect」→「網路連結」→「psc-network-attachment」

The screenshot shows the Google Cloud console interface. On the left, the 'Network Services' sidebar is visible, with 'Private Service Connect' highlighted. The main content area is titled 'Network attachment details' and shows the configuration for the 'psc-network-attachment'. The configuration table lists the following details:

Property	Value
Network attachment	projects/pipelines-validation/regions/us-central1/networkAttachments/psc-network-attachment
Region	us-central1
Network	consumer-vpc
Subnetworks	intf-subnet
Connection preference	Accept manually

Below the configuration table, there is a section for 'Connected projects'.

步驟 7 · 約 3 分鐘

7. 私人 DNS 區域

您將為 demo.com 建立 Cloud DNS 區域，並填入指向 VM IP 位址的 A 記錄。稍後，DNS 對等互連會部署在 Vertex AI Pipelines 工作中，以便存取消費者的 DNS 記錄。

在 Cloud Shell 中執行下列操作：

```
gcloud dns --project=$projectid managed-zones create private-dns-codelab --description="" --
dns-name="demo.com." --visibility="private" --
networks="https://compute.googleapis.com/compute/v1/projects/$projectid/global/networks/consumer-vpc"
```

在 Cloud Shell 中，對 VM 執行個體執行說明，取得各自的 IP 位址。

```
gcloud compute instances describe class-e-vm --zone=us-central1-a | grep networkIP:

gcloud compute instances describe proxy-vm --zone=us-central1-a | grep networkIP:
```

在 Cloud Shell 中，為 VM (class-e-vm) 建立記錄集，並根據環境的輸出內容更新 IP 位址。

```
gcloud dns --project=$projectid record-sets create class-e-vm.demo.com. --zone="private-dns-
codelab" --type="A" --ttl="300" --rrdatas="240.0.0.2"
```

在 Cloud Shell 中，為 VM (proxy-vm) 建立記錄集，並根據環境的輸出內容更新 IP 位址。

```
gcloud dns --project=$projectid record-sets create proxy-vm.demo.com. --zone="private-dns-
codelab" --type="A" --ttl="300" --rrdatas="10.10.10.2"
```

建立 Cloud Firewall 防火牆規則，允許從 PSC 介面存取

在下一節中，建立防火牆規則，允許來自 PSC 網路附件的流量存取消費者虛擬私有雲中的 RFC1918 計算資源。

在 Cloud Shell 中，建立允許從 PSC 網路附件子網路存取 proxy-vm 的輸入防火牆規則。

```
gcloud compute firewall-rules create allow-access-to-proxy \
  --network=consumer-vpc \
  --action=ALLOW \
  --rules=ALL \
  --direction=INGRESS \
  --priority=1000 \
  --source-ranges="192.168.10.0/28" \
  --destination-ranges="10.10.0.0/19" \
  --enable-logging
```

在 Cloud Shell 中，建立允許從 proxy-vm 子網路存取 class-e 子網路的輸入防火牆規則。

```
gcloud compute firewall-rules create allow-access-to-class-e \  
  --network=consumer-vpc \  
  --action=ALLOW \  
  --rules=ALL \  
  --direction=INGRESS \  
  --priority=1000 \  
  --source-ranges="10.10.10.0/28" \  
  --destination-ranges="240.0.0.0/4" \  
  --enable-logging
```

8. 更新明確 Proxy

在下一節中，您需要透過 SSH 連線至明確的 Proxy，更新 `tinyproxy.conf` 設定檔，然後執行重設。

透過 Cloud Shell

```
gcloud compute ssh --zone us-central1-a "proxy-vm" --tunnel-through-iap --project $projectId
```

開啟 `tinyproxy` 設定檔，使用編輯器或您選擇的工具更新。以下是使用 VIM 的範例。

```
sudo vim /etc/tinyproxy/tinyproxy.conf

# Locate the "Listen" configuration line to restrict listening to only its private IP address
of the Proxy-VM, rather than all interfaces.

Listen 10.10.10.2

# Locate the "Allow" configuration line to allow requests ONLY from the PSC Network
Attachment Subnet

Allow 192.168.10.0/24

Save the configs by the following steps:
1. Press the `ESC` key to enter Command Mode.
2. Type `:wq` to save (w) and quit (q).
3. Press `Enter`

Restart the tinyproxy service to apply the changes:
sudo systemctl restart tinyproxy

Validate the tinyproxy service is running:
sudo systemctl status tinyproxy

Perform an exit returning to cloud shell
exit
```

9. 建立 Jupyter Notebook

下一節將引導您建立 Jupyter Notebook。這個筆記本會用於部署 Vertex AI Pipelines 工作，將 wget 從 Vertex AI Pipelines 傳送至測試執行個體。Vertex AI Pipelines 與含有執行個體的消費者網路之間的資料路徑，會使用 Private Service Connect 網路介面。

建立使用者管理的服務帳戶

在下一節中，您將建立與本教學課程所用 Vertex AI Workbench 執行個體相關聯的服務帳戶。

在本教學課程中，服務帳戶會套用下列角色：

- [儲存空間管理員](#)
- [Vertex AI 使用者](#)
- [Artifact Registry 管理員](#)
- [Cloud Build 編輯者](#)
- [IAM 服務帳戶使用者](#)

在 Cloud Shell 中建立服務帳戶。

```
gcloud iam service-accounts create notebook-sa \  
  --display-name="notebook-sa"
```

在 Cloud Shell 中，將服務帳戶更新為 Storage 管理員角色。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:notebook-  
sa@$projectid.iam.gserviceaccount.com" --role="roles/storage.admin"
```

在 Cloud Shell 中，使用 Vertex AI 使用者角色更新服務帳戶。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:notebook-  
sa@$projectid.iam.gserviceaccount.com" --role="roles/aiplatform.user"
```

在 Cloud Shell 中，更新服務帳戶，並指派 Artifact Registry 管理員角色。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:notebook-  
sa@$projectid.iam.gserviceaccount.com" --role="roles/artifactregistry.admin"
```

在 Cloud Shell 中，將 Cloud Build 編輯者角色指派給服務帳戶。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:notebook-  
sa@$projectid.iam.gserviceaccount.com" --role="roles/cloudbuild.builds.editor"
```

在 Cloud Shell 中，允許筆記本服務帳戶使用 Compute Engine 預設服務帳戶。

```
gcloud iam service-accounts add-iam-policy-binding \  
    $(gcloud projects describe $(gcloud config get-value project) --  
format='value(projectNumber)')-compute@developer.gserviceaccount.com \  
    --member="serviceAccount:notebook-sa@$projectid.iam.gserviceaccount.com" \  
    --role="roles/iam.serviceAccountUser"
```

步驟 10 · 約 5 分鐘

10. 建立 Vertex AI Workbench 執行個體

在下一節中，建立納入先前建立的服務帳戶 notebook-sa 的 Vertex AI Workbench 執行個體。

在 Cloud Shell 中建立 private-client 執行個體。

```
gcloud workbench instances create workbench-tutorial --vm-image-project=cloud-notebooks-managed --vm-image-family=workbench-instances --machine-type=n1-standard-4 --location=us-central1-a --subnet-region=us-central1 --subnet=rfc1918-subnet1 --disable-public-ip --shielded-secure-boot=true --shielded-integrity-monitoring=true --shielded-vtpm=true --service-account-email=notebook-sa@$projectid.iam.gserviceaccount.com
```

11. 更新 Vertex AI 服務代理

Vertex AI 會代表您執行作業，例如從用於建立 PSC 介面的 PSC 網路連結子網路取得 IP 位址。為此，Vertex AI 會使用[服務代理](#) (如下所列)，該代理需要[網路管理員](#)權限。

`service-$projectnumber@gcp-sa-aiplatform.iam.gserviceaccount.com`

注意：更新服務代理程式權限前，請先前往 Cloud 控制台的 Vertex AI，確認已啟用 Vertex AI API。

在 Cloud Shell 中取得專案編號。

```
gcloud projects describe $projectid | grep projectNumber
```

在 Cloud Shell 中取得專案編號。

```
gcloud projects describe $projectid | grep projectNumber
projectNumber: '234086459238'
```

在 Cloud Shell 中設定專案編號。

```
projectnumber=YOUR-PROJECT-Number
```

在 Cloud Shell 中，為 AI Platform 建立服務帳戶。如果專案中已有服務帳戶，請略過這個步驟。

```
gcloud beta services identity create --service=aiplatform.googleapis.com --
project=$projectnumber
```

在 Cloud Shell 中，使用 `compute.networkAdmin` 角色更新服務代理人帳戶。

```
gcloud projects add-iam-policy-binding $projectid --
member="serviceAccount:service-$projectnumber@gcp-sa-aiplatform.iam.gserviceaccount.com" --
role="roles/compute.networkAdmin"
```

在 Cloud Shell 中，使用 `dns.peer` 角色更新服務代理程式帳戶

```
gcloud projects add-iam-policy-binding $projectid --
member="serviceAccount:service-$projectnumber@gcp-sa-aiplatform.iam.gserviceaccount.com" --
role="roles/dns.peer"
```

更新預設服務帳戶

啟用 Compute Engine API，並[授予預設服務帳戶](#) Vertex AI 存取權。請注意，存取權變更可能需要一段時間才會生效。

在 Cloud Shell 中，使用 `aiplatform.user` 角色更新預設服務帳戶

```
gcloud projects add-iam-policy-binding $projectid \  
  --member="serviceAccount:$projectnumber-compute@developer.gserviceaccount.com" \  
  --role="roles/aiplatform.user"
```

在 Cloud Shell 中，使用 storage.admin 角色更新預設服務帳戶

```
gcloud projects add-iam-policy-binding $projectid \  
  --member="serviceAccount:$projectnumber-compute@developer.gserviceaccount.com" \  
  --role="roles/storage.admin"
```

在 Cloud Shell 中，使用 storage.admin 角色更新預設服務帳戶

```
gcloud projects add-iam-policy-binding $projectid \  
  --member="serviceAccount:$projectnumber-compute@developer.gserviceaccount.com" \  
  --role="roles/artifactregistry.admin"
```

12. 啟用 Tcpdump

如要驗證 Vertex AI Pipelines 的 IP 連線，可以使用 TCPDUMP。這樣一來，當我們從 Vertex AI Pipelines 叫用對 vm 的 get 請求時，就能觀察源自 PSC 網路附件子網路 (192.168.10.0/28) 的通訊，class-e-vm.demo.com (240.0.0.0/4)。

從 Cloud Shell 透過 SSH 連線至 Proxy VM。

```
gcloud compute ssh --zone us-centrall1-a "proxy-vm" --tunnel-through-iap --project $projectid
```

從 proxy-vm OS 執行 tcpdump，並在 class-e-vm 和 PSC 網路附加子網路上篩選。

```
sudo tcpdump -i any net 240.0.0.0/4 or 192.168.10.0/28 -nn
```

開啟新的 Cloud Shell 分頁，更新專案變數，然後透過 SSH 連線至 class-e-vm

```
gcloud compute ssh --zone us-centrall1-a "class-e-vm" --tunnel-through-iap --project $projectid
```

從 class-e-vm OS 執行 tcpdump，篩選 proxy-vm 子網路。

```
sudo tcpdump -i any net 10.10.10.0/28 -nn
```

13. 部署 Vertex AI Pipelines 工作

在下一節中，您將建立筆記本，從 Vertex AI Pipelines 成功執行 wget 至明確的 Proxy。這樣一來，您就能連線至非 RFC 1918 VM，例如 class-e-vm。由於目標是 RFC 1918 IP 位址，Vertex AI Pipelines 無須明確的 Proxy 即可存取 rfc1918-vm。

在 Vertex AI Workbench 執行個體中執行訓練工作。

1. 在 Google Cloud 控制台中，前往 Vertex AI Workbench 頁面的執行個體分頁。
2. 按一下 Vertex AI Workbench 執行個體名稱 (workbench-tutorial) 旁的「Open JupyterLab」。Vertex AI Workbench 執行個體會在 JupyterLab 中開啟。
3. 依序選取「檔案」>「新增」>「筆記本」
4. 選取「Kernel」>「Python 3」

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
# Install gcloud
!pip install google-cloud

# Install the pipeline required packages
!pip install --upgrade google-cloud-aiplatform \
                google-cloud-storage \
                kfp \
                google-cloud-pipeline-components

# Import libraries
from time import gmtime, strftime
import json
import requests
```

在 JupyterLab 筆記本中建立新儲存格，然後更新並執行下列項目。請務必將 PROJECT_ID 更新為環境詳細資料。

```
import json
import requests
import pprint

PROJECT_ID = 'YOUR-PROJECT-ID' #Enter your project ID
PROJECT_NUMBER=!gcloud projects list --filter="project_id:$PROJECT_ID" --
format="value(PROJECT_NUMBER)"
PROJECT_NUMBER=str(PROJECT_NUMBER).strip('[').strip(']').strip('"')
print(PROJECT_NUMBER)
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
# us-central1 is used for the codelab
REGION = "us-central1" #@param {type:"string"}
SERVICE_NAME = "aiplatform" #@param {type:"string"}
SERVICE ="{}.googleapis.com".format(SERVICE_NAME)
ENDPOINT="{}-{}.googleapis.com".format(REGION, SERVICE_NAME)
API_VERSION = "v1" # @param {type: "string"}

LOCATION = REGION
```

在 JupyterLab 筆記本中建立新儲存格，然後執行下列設定，並注意以下重點：

- proxy_server = "http://proxy-vm.demo.com:8888" FQDN 與部署在消費者 VPC 中的 Proxy VM 相關聯。我們會在後續步驟中使用 DNS 對等互連來解析 FQDN。

```

%%writefile main.py

import logging
import socket
import sys
import os

def make_api_request(url: str, proxy_vm_ip: str, proxy_vm_port: str):
    """
    Makes a GET request to a non-rfc1918 API and saves the response.

    Args:
        url: The URL of the API to send the request to.
    """
    import requests

    try:
        # response = requests.get(url)
        proxy_server = f"http://proxy-vm.demo.com:8888" # replace with you VM's IP and proxy
        port.

        proxies = {
            "http": proxy_server,
            "https": proxy_server,
        }

        response = requests.get(url, proxies=proxies)
        logging.info(response.text)

        response.raise_for_status() # Raise an exception for bad status codes
        logging.info(f"Successfully fetched data from {url}")
    except requests.exceptions.RequestException as e:
        logging.error(f"An error occurred: {e}")
        raise e

if __name__ == '__main__':
    # Configure logging to print clearly to the console
    logging.basicConfig(
        level=logging.INFO,
        format='%(levelname)s: %(message)s',
        stream=sys.stdout
    )
    url_to_test = os.environ['NONRFC_URL']
    proxy_vm_ip = os.environ['PROXY_VM_IP']
    proxy_vm_port = os.environ['PROXY_VM_PORT']

    logging.info(f"url_to_test: {url_to_test}")
    logging.info(f"proxy_vm_ip: {proxy_vm_ip}")
    logging.info(f"proxy_vm_port: {proxy_vm_port}")
    make_api_request(url_to_test, proxy_vm_ip, proxy_vm_port)

```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
%%writefile Dockerfile
FROM python:3.9-slim

RUN apt-get update && \
    apt-get install -y iputils-ping && \
    apt-get install -y wget

RUN pip install cloudml-hypertune requests kfp

COPY main.py /main.py

ENTRYPOINT ["python3", "/main.py"]
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
!gcloud artifacts repositories create pipelines-test-repo-psc --repository-format=docker --
location=us-central1
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
IMAGE_PROJECT = PROJECT_ID
IMAGE_REPO = 'pipelines-test-repo-psc'
IMAGE_NAME = 'nonrfc-ip-call'
TAG = 'v1'

IMAGE_URI= f'us-central1-docker.pkg.dev/{IMAGE_PROJECT}/{IMAGE_REPO}/{IMAGE_NAME}:{TAG}'
IMAGE_URI
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
!gcloud auth configure-docker us-docker.pkg.dev --quiet
```

在 JupyterLab 筆記本中，建立新的儲存格並執行下列程式碼。如有錯誤 (gcloud.builds.submit)，請忽略。

```
!gcloud builds submit --tag {IMAGE_URI} --region=us-central1
```

在 JupyterLab 筆記本中建立並執行下列儲存格，請注意以下重點：

- 使用網域名稱 demo.com 的 dnsPeeringConfigs (dnsPeeringConfigs)，設定與消費者虛擬私有雲的 DNS 對等互連。
- 明確的 Proxy (定義為 PROXY_VM_IP 變數) 是 proxy-vm.demo.com。解析作業會在用戶的 VPC 內透過 DNS 對接處理。
- 通訊埠 8888 是 tinyproxy 中設定的監聽通訊埠 (預設)
- 透過 DNS 對等互連解析 wget 至 class-e-vm-demo.com
- 程式碼會為 Vertex 指定「psc-network-attachment」，讓 Vertex 能使用網路連結子網路部署兩個 PSC 介面執行個體。

```

import json
from datetime import datetime

JOB_ID_PREFIX='test_psci-nonRFC' #@param {type:"string"}
JOB_ID = '{}_{}'.format(JOB_ID_PREFIX, datetime.now().strftime("%Y%m%d%H%M%S"))

# PSC-I configs

PRODUCER_PROJECT_ID = PROJECT_ID
DNS_DOMAIN = 'class-e-vm.demo.com' #@param {type:"string"}
NON_RFC_URL = f"http://{DNS_DOMAIN}"

PROXY_VM_IP = "proxy-vm.demo.com" #@param {type:"string"}
PROXY_VM_PORT = "8888" #@param {type:"string"}

CUSTOM_JOB = {
    "display_name": JOB_ID,
    "job_spec": {
        "worker_pool_specs": [
            {
                "machine_spec": {
                    "machine_type": "n1-standard-4",
                },
                "replica_count": 1,
                "container_spec": {
                    "image_uri": IMAGE_URI,
                    "env": [{
                        "name": "NONRFC_URL",
                        "value": NON_RFC_URL
                    },
                    {
                        "name": "PROXY_VM_IP",
                        "value": PROXY_VM_IP
                    },
                    {
                        "name": "PROXY_VM_PORT",
                        "value": PROXY_VM_PORT
                    }
                ]
            },
        ],
        "enable_web_access": True,
        "psc_interface_config": {
            "network_attachment": "psc-network-attachment",
            "dns_peering_configs": [
                {
                    "domain": "demo.com.",
                    "target_project": PROJECT_ID,
                    "target_network": "consumer-vpc"
                },
            ],
        ],
    },
}

```



```

    },
}
}

print(json.dumps(CUSTOM_JOB, indent=2))

```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```

import requests
bearer_token = !gcloud auth application-default print-access-token
headers = {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer {}'.format(bearer_token[0]),
}

request_uri = f"https://{REGION}-
aiplatform.googleapis.com/{API_VERSION}/projects/{PROJECT_NUMBER}/locations/{REGION}/customJobs/"

print("request_uri: ", request_uri)

```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```

response_autopush = requests.post(request_uri, json=CUSTOM_JOB, headers=headers)
response = response_autopush
print("response:", response)
if response.reason == 'OK':
    job_name = response.json()['name']
    job_id = job_name.split('/')[-1]
    print("Created Job: ", response.json()['name'])
else:
    print(response.text)

```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```

# Print KFP SDK version (should be >= 1.6)
! python3 -c "import kfp; print('KFP SDK version: {}'.format(kfp.__version__))"

# Print AI Platform version
! python3 -c "from google.cloud import aiplatform; print('AI Platform version:
{}'.format(aiplatform.__version__))"

```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```

BUCKET_URI = "your-unique-bucket" # Provide a globally unique bucket name

```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
!gcloud storage buckets create gs://{BUCKET_URI}
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
# pipeline parameters
CACHE_PIPELINE = False # @param {type: "string"}
_DEFAULT_IMAGE = IMAGE_URI
BUCKET_URI = "gs://{BUCKET_URI}" # @param {type: "string"}
PIPELINE_ROOT = f"{BUCKET_URI}/pipeline_root/intro"
PIPELINE_DISPLAY_NAME = "pipeline_nonRFCIP" # @param {type: "string"}
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```

from re import S
import kfp
from kfp import dsl
from kfp.dsl import container_component, ContainerSpec
from kfp import compiler
from google.cloud import aiplatform

# ==== Component with env variable ====

@container_component
def dns_peering_test_op(dns_domain: str, proxy_vm_ip:str, proxy_vm_port:str):
    return ContainerSpec(
        image=_DEFAULT_IMAGE,
        command=["bash", "-c"],
        args=[
            """
            apt-get update && apt-get install inetutils-traceroute inetutils-ping netcat-
openbsd curl -y

            echo "Local IP(s): $(hostname -I)"

            echo "Attempting to trace route to %s"
            traceroute -w 1 -m 7 "%s"

            echo "Sending curl requests to http://%s via proxy %s:%s and recording trace..."
            if curl -L -v --trace-ascii /dev/stdout -x http://%s:%s "http://%s"; then
                echo "Curl request succeeded!"
            else
                echo "Curl request failed!"
                exit 1
            fi
            """ % (dns_domain, dns_domain, dns_domain, proxy_vm_ip, proxy_vm_port,
proxy_vm_ip, proxy_vm_port, dns_domain)

        ]
    )

# ==== Pipeline ====
@dsl.pipeline(
    name="dns-peering-test-pipeline",
    description="Test DNS Peering using env variable",
    pipeline_root=PIPELINE_ROOT,
)
def dns_peering_test_pipeline(dns_domain: str, proxy_vm_ip:str, proxy_vm_port:str):
    dns_test_task = dns_peering_test_op(dns_domain=dns_domain, proxy_vm_ip=proxy_vm_ip,
proxy_vm_port=proxy_vm_port)
    dns_test_task.set_caching_options(enable_caching=CACHE_PIPELINE)

# ==== Compile pipeline ====
if __name__ == "__main__":
    aiplatform.init(project=PROJECT_ID, location=LOCATION)

```

```
compiler.Compiler().compile(  
    pipeline_func=dns_peering_test_pipeline,  
    package_path="dns_peering_test_pipeline.yaml",  
)  
print("✅ Pipeline compiled to dns_peering_test_pipeline.yaml")
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
# Define the PipelineJob body; see API Reference https://cloud.google.com/vertex-ai/docs/reference/rest/v1/projects.locations.pipelineJobs/create  
  
import requests, json  
import datetime  
  
bearer_token = !gcloud auth application-default print-access-token  
headers = {  
    'Content-Type': 'application/json',  
    'Authorization': 'Bearer {}'.format(bearer_token[0]),  
}  
  
request_uri = f"https://{REGION}-  
aiplatform.googleapis.com/{API_VERSION}/projects/{PROJECT_NUMBER}/locations/{REGION}/pipeline  
Jobs/"  
  
print("request_uri: ", request_uri)
```

注意：首次執行時，最後一個儲存格執行完畢後，Vertex AI Pipelines 訓練作業可能需要最多 15 分鐘才能完成。如要監控狀態，請前往下列位置：

Vertex AI → 「訓練」 → 「自訂工作」

14. PSC 介面驗證

您也可以前往下列位置，查看 Vertex AI Pipelines 使用的網路附件 IP：

「網路服務」 → 「Private Service Connect」 → 「網路連結」 → 「psc-network-attachment」

選取租戶專案 (專案名稱結尾為「-tp」)

psc-network-attachment

Network attachment	projects/vertex-cosmo/regions/us-central1/networkAttachments/psc-network-attachment 
Region	us-central1
Network	consumer-vpc
Subnetworks	intf-subnet
Connection preference	Accept manually

Connected projects

 Filter	Filter projects			
☐	Name ↑	Status	Connections	Actions
☐	fc04d8d4168daeb73-tp	✔ Accepted	2	⋮

醒目顯示的欄位表示 Vertex AI Pipelines 從 PSC 網路連結使用的 IP 位址。

fc04d8d4168daeb73-tp

Status	✔ Accepted
Connections	2

Endpoints

 Filter	Filter endpoints	
Subnetwork ↑	IPv4 address	
intf-subnet	192.168.10.4	
intf-subnet	192.168.10.5	

15. Cloud Logging 驗證

Vertex AI Pipelines 工作首次執行約需 14 分鐘，後續執行時間會大幅縮短。如要驗證結果是否成功，請執行下列操作：

前往「Vertex AI」→「訓練」→「自訂工作」

選取已執行的自訂工作

Training

+

Train new model

↺

Refresh

Training pipelines

Custom jobs

Hyperparameter tuning jobs

NAS jobs

P

Custom jobs specify how Vertex AI runs your custom training code, including worker pools, machine types, and settings related to your Python training application and custom container. Custom jobs are only used by custom-trained models and not AutoML models. [Learn more](#)

Region

us-central1 (Iowa)

Filter

Enter a property name

Name	ID	Status
test_psci-nonRFC_20250912025943	6594373987482992640	✔ Finished

選取「查看記錄」

Status	Finished
Custom job ID	6594373987482992640
Created	Sep 11, 2025, 10:00:13 PM
Start time	Sep 11, 2025, 10:06:01 PM
Region	us-central1
Encryption type	Google-managed
Machine type (Worker pool 0 (chief))	n1-standard-4
Machine type (Worker pool 0 (chief))	1
Container Location (Worker pool 0 (chief))	us-central1-docker.pkg.dev/vertex-cosmo/pipelines-test-repo-psc/nonrfc-ip-call:v1
Algorithm	Custom training
Objective	Custom
Container (Training)	Custom
Logs	View logs

Cloud Logging 可用後，請選取「Run Query」，產生下方醒目顯示的選取項目，確認從 Vertex AI Pipelines 到 class-e-vm 的 wget 成功。

Logs Explorer

Query library

Share link

Preferences

unavailable

Learn

Project logs

Search all fields

All resources

All log names

All severities

Correlate by

+2 filters

1

resource.labels.job_id="6594373987482992640" timestamp="2025-09-12T03:00:13.075504Z"

Run query

Show query

SEVERITY	TIME	SUMMARY
 Showing logs for time specified in query. To view more results update your query.		
> i	2025-09-11 22:02:25.116	service Waiting for job to be provisioned.
> i	2025-09-11 22:02:25.130	service Vertex AI is provisioning job running framework. First time usage might take couple of minutes, and subsequent runs can be much faster.
> i	2025-09-11 22:02:25.184	service Vertex AI is setting up this job.
> i	2025-09-11 22:02:25.184	service Waiting for training program to start.
> i	2025-09-11 22:02:27.476	service Job is preparing.
> i	2025-09-11 22:05:53.382	workerpool0-0 INFO: url_to_test: http://class-e-vm.demo.com
> i	2025-09-11 22:05:53.382	workerpool0-0 INFO: proxy_vm_ip: proxy-vm.demo.com
> i	2025-09-11 22:05:53.382	workerpool0-0 INFO: proxy_vm_port: 8888
> i	2025-09-11 22:05:53.382	workerpool0-0 INFO: Class-e server !!
> i	2025-09-11 22:05:53.382	workerpool0-0 {"levelname":"INFO", "message":""}
> i	2025-09-11 22:05:53.382	workerpool0-0 INFO: Successfully fetched data from http://class-e-vm.demo.com
> i	2025-09-11 22:06:01.487	service Job is running.
> i	2025-09-11 22:06:01.637	service Tearing down training program.
> i	2025-09-11 22:09:42.948	service Finished tearing down training program.
> i	2025-09-11 22:09:43.278	service Job completed successfully.

16. TCPDump 驗證

讓我們來檢查 TCPDUMP 輸出內容，進一步驗證與運算執行個體的連線：

從 proxy-vm 觀察 HTTP GET 和 200 OK

```
03:05:34.778574 ens4 Out IP 10.10.10.2.40326 > 240.0.0.2.80: Flags [P.], seq 1:63,
ack 1, win 511, options [nop,nop,TS val 1435446009 ecr 2475360885], length 62: HTTP:
GET / HTTP/1.0
03:05:34.778946 ens4 In IP 240.0.0.2.80 > 10.10.10.2.40326: Flags [.], ack 63, win
506, options [nop,nop,TS val 2475360889 ecr 1435446009], length 0
03:05:34.778974 ens4 Out IP 10.10.10.2.40326 > 240.0.0.2.80: Flags [P.], seq 63:185,
ack 1, win 511, options [nop,nop,TS val 1435446010 ecr 2475360889], length 122: HTTP
03:05:34.781999 ens4 In IP 240.0.0.2.80 > 10.10.10.2.40326: Flags [.], ack 185, win
506, options [nop,nop,TS val 2475360892 ecr 1435446010], length 0
03:05:34.906678 ens4 In IP 240.0.0.2.80 > 10.10.10.2.40326: Flags [P.], seq 1:265,
ack 185, win 506, options [nop,nop,TS val 2475361016 ecr 1435446010], length 264:
HTTP: HTTP/1.1 200 OK
```

從 class-e-vm 觀察 HTTP GET 和 200 OK

```
03:05:34.778768 ens4 In IP 10.10.10.2.40326 > 240.0.0.2.80: Flags [P.], seq 1:63,
ack 1, win 511, options [nop,nop,TS val 1435446009 ecr 2475360885], length 62: HTTP:
GET / HTTP/1.0
03:05:34.778819 ens4 Out IP 240.0.0.2.80 > 10.10.10.2.40326: Flags [.], ack 63, win
506, options [nop,nop,TS val 2475360889 ecr 1435446009], length 0
03:05:34.781815 ens4 In IP 10.10.10.2.40326 > 240.0.0.2.80: Flags [P.], seq 63:185,
ack 1, win 511, options [nop,nop,TS val 1435446010 ecr 2475360889], length 122: HTTP
03:05:34.781856 ens4 Out IP 240.0.0.2.80 > 10.10.10.2.40326: Flags [.], ack 185, win
506, options [nop,nop,TS val 2475360892 ecr 1435446010], length 0
03:05:34.906503 ens4 Out IP 240.0.0.2.80 > 10.10.10.2.40326: Flags [P.], seq 1:265,
ack 185, win 506, options [nop,nop,TS val 2475361016 ecr 1435446010], length 264:
HTTP: HTTP/1.1 200 OK
```


17. 清理

注意：Vertex AI Pipeline 至少一小時未使用後，您即可刪除 PSC 網路連結和子網路。

在 Cloud Shell 中刪除教學課程元件。

```
gcloud compute instances delete proxy-vm --zone=us-central1-a --quiet

gcloud compute instances delete workbench-tutorial --zone=us-central1-a --quiet

gcloud compute routers delete cloud-router-for-nat --region=us-central1 --quiet

gcloud compute network-attachments delete psc-network-attachment --region=us-central1 --quiet

gcloud compute networks subnets delete intf-subnet rfc1918-subnet1 --region=us-central1 --quiet

gcloud dns record-sets delete class-e-vm.demo.com --zone=private-dns-codelab --type=A
gcloud dns record-sets delete proxy-vm.demo.com --zone=private-dns-codelab --type=A

gcloud dns managed-zones delete private-dns-codelab
gcloud compute networks delete consumer-vpc --quiet
```

步驟 18 · 約 0 分鐘

18. 恭喜

恭喜！您已成功設定並驗證 Private Service Connect 介面與 Vertex AI Pipelines 的連線。

您已建立消費者基礎架構，並新增網路附件，讓生產者可以建立多重 NIC VM，以橋接消費者和生產者通訊。您已瞭解如何建立 DNS 對等互連，同時在用戶虛擬私有雲網路中部署明確的 Proxy，允許連線至無法直接從 Vertex 路由傳送的 class-e-vm 執行個體。

Cosmopup 認為教學課程很棒！



後續步驟

延伸閱讀和影片

- [Private Service Connect 總覽](#)

What is Private Service Connect?

Google Cloud Tech

Private Service Connect

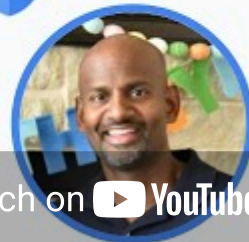


Watch on YouTube

Vertex AI networking patterns - PSC & Google APIs demo

Advanced Networking Demo

Connect to Vertex AI using PSC & Googleapis



Watch on YouTube

- [關於 Private Service Connect 介面 | VPC | Google Cloud](#)

參考文件

- [Vertex AI 網路存取權總覽 | Google Cloud](#)
- [透過 Private Service Connect 介面存取 Vertex AI 服務 | Google Cloud](#)
- [使用 Vertex AI 訓練 的 Private Service Connect 介面 | Google Cloud](#)
- [為 Vertex AI 資源設定 Private Service Connect 介面 | Google Cloud](#)